Parameter passing in NWChem and the Runtime Database

Rohit Goswami · 2025-08-16 · hpc · science · components · parameters · nwchem

> *Thoughts on parameter passing through the RTDB within NWChem.*

Background

Despite a plethora of publications (Aprà et al. 2020; Mejia-Rodriguez et al. 2023), NWChem has little to nothing in terms of developer documentation [^fn:1]. For better integration over sockets with EON (described elsewhere), I ended up having to add user defined parameters, from which this short note stems.

Inputs in NWChem

The input file format in NWChem, like most codes of the era, consist of a keyword driven, whitespace delimited, block structured set of directives which are parsed once into a database and propagated across other modules.

Runtime Database (RTDB)

The run time database, as noted in (Bernholdt et al. 1995; Guest et al. 1996), is similar to the GAMESS dump file or GAUSSIAN checkpoint file. Allegedly it is easier and safer than both, and arrays of typed data are stored with ASCII strings for keys. Though the GAMESS and GAUSSIAN notes are not an immediately recognizable point of reference either, but it is essentially a key value store which is used to circumvent having an interoperable data-structure.

This centralized key-value approach decouples modules from the specifics of the input file; a routine only needs to know the key for a parameter, not how or where it was parsed. This contrasts with approaches where configuration objects must be passed through long call stacks or where multiple components parse the same configuration file independently.

Adding parameters to NWChem through the RTDB

Here, we will only be interested in this section:

```
driver
  socket unix eon_nwchem
end
```

Which can be abstracted into:

```
driver
  socket (unix || ipi_client) <socketname>
end
```

Concretely, we will consider the case of adding optional parameters to this so as to enable the following design:

```
driver
  socket (unix || ipi_client) <socketname> [retries <int>] [delay <int>]
end
```

Conceptually this involves:

Parsing

To extract data from the input

Storing

Committing parsed values into the RTDB with `rtdb_put`

Retrieving

Accessing the values from the RTDB with `rtdb_get`

Note
The complete changeset required is enumerated in the [pull request](https://github.com/nwchemgit/nwchem/pull/1145) with a documentation update to the wiki [here](https://github.com/nwchemgit/nwchem-wiki/pull/39).

Parsing optional keyword arguments

Naming follows a conventional scheme[^fn:2], so the `src/driver/driver_input.F` is the first port of call. The implementation involves setting defaults and then looping through remaining fields on the line to check for overrides.

```
! socket ipi_client ip:port [retries <int>] [delay <int>]
max_retries = 30
retry_delay = 2
do while (inp_a(field))
   if (inp_compare(.false., 'retries', field)) then
      if (.not. inp_i(max_retries))
      call errquit('driver_input: expected integer for retries', 0,INPUT_ERR)
   else if (inp_compare(.false., 'delay', field)) then
      if (.not. inp_i(retry_delay))
      call errquit('driver_input: expected int for delay',0,INPUT_ERR)
   end if
end do
```

Where `inp_a` parses a keyword, `inp_compare` checks against an expected value, and `inp_i` parses the value as an integer.

Committing values to the RTDB

Once stored in a local variable, the value is then committed to the RTDB using `rtdb_put`, which is prefixed into a namespace for modularity and to reduce key conflicts.

```
if (inp_compare(.false.,'retries',field)) then
  if (.not. inp_i(max_retries)) call errquit(...)
    if (.not. rtdb_put(rtdb,'driver:socket_retries',mt_int,1,max_retries)) call errquit(...)
end if
```

Where the signature notes the number of elements (`1`) being stored along with the type (`mt_int`).

Retrieval and usage

Where the values are to be retrieved, say, here in `src/driver/socket_driver.F`, defaults are used if the key is missing.

```
if (.not. rtdb_get(rtdb, 'driver:socket_retries', mt_int, 1, max_retries)) then
   max_retries = 30
end if
```

Before being used as needed. The signature of `rtdb_get` conceptually mimics that of `rtdb_put`. In practice, since these are FORTRAN 77 files in `nwchem`, there are a few more continuation line characters and other marginalia, but otherwise this is all that is necessary to expose user parameters to modules in NWChem.

Conclusions

Although not the design taken by more modern software, the RTDB is pretty intuitive to work with, and probably better than GAMESS, GAUSSIAN, or even the bring your own data into a simulation development environment idea of NWChemEx (Richard et al. 2019). Better documentation would probably stemmed the split. Modern tools (e.g. EON) improve upon this by using more structured input files, e.g. using TOML ensures typing which can then be independently read into C/C++/Fortran etc. though the issue of consistency at runtime remains.

Reading list

NWChem, its runtime database, and the surrounding work on scientific software design, collected as a citation list: 6 references. The collection exports to BibTeX or any CSL style, so it can go straight into a bibliography.

References

Aprà, E., E. J. Bylaska, W. A. De Jong, N. Govind, K. Kowalski, T. P. Straatsma, M. Valiev, et al. 2020. "NWChem: Past, Present, and Future." *Journal of Chemical Physics* 152 (18): 184102. <https://doi.org/10.1063/5.0004997>.

Bernholdt, D. E., E. Apr, H. A. Frchtl, M. F. Guest, R. J. Harrison, R. A. Kendall, R. A. Kutteh, et al. 1995. "Parallel Computational Chemistry Made Easier: The Development of NWChem." *International Journal of Quantum Chemistry* 56 (S29): 475--83. <https://doi.org/10.1002/qua.560560851>.

Guest, M. F., E. Apra, D. E. Bernholdt, H. A. Früchtl, R. J. Harrison, R. A. Kendall, R. A. Kutteh, et al. 1996. "High-Performance Computing in Chemistry: NW Chem." *Future Generation Computer Systems* 12 (4): 273--89. <https://doi.org/10.1016/S0167-739X(97)80002-E>.

Kowalski, Karol, Raymond Bair, Nicholas P. Bauman, Jeffery S. Boschen, Eric J. Bylaska, Jeff Daily, Wibe A. De Jong, et al. 2021. "From NWChem to NWChemEx: Evolving with the Computational Chemistry Landscape." *Chemical Reviews* 121 (8): 4962--98. <https://doi.org/10.1021/acs.chemrev.0c00998>.

Mejia-Rodriguez, Daniel, Edoardo Aprà, Jochen Autschbach, Nicholas P. Bauman, Eric J. Bylaska, Niranjan Govind, Jeff R. Hammond, et al. 2023. "NWChem: Recent and Ongoing Developments." *Journal of Chemical Theory and Computation* 19 (20): 7077--96. <https://doi.org/10.1021/acs.jctc.3c00421>.

Richard, Ryan M., Colleen Bertoni, Jeffery S. Boschen, Kristopher Keipert, Benjamin Pritchard, Edward F. Valeev, Robert J. Harrison, Wibe A. De Jong, and Theresa L. Windus. 2019. "Developing a Computational Chemistry Framework for the Exascale Era." *Computing in Science & Engineering* 21 (2): 48--58. <https://doi.org/10.1109/MCSE.2018.2884921>.

```
et al. 2021](#ref-kowalskiNWChemNWChemExEvolving2021)).
```
